

SET-A



Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [20 Points]

Write a Java program that takes an integer **N** as input, where **N contains at least 6 digits**.

- Check the **first (leftmost) digit** of the number.
- If the first digit is **even**, print **only the even digits** of the number from **left to right**.
- Otherwise, print **only the odd digits** of the number from **left to right**.

Print the selected digits separated by a single space.

Sample Input	Sample Output	Explanation
82461357	8 2 4 6	The first digit is 8 , which is even. Therefore, all even digits are printed from left to right.
73184295	7 3 1 9 5	The first digit is 7 , which is odd. Therefore, all odd digits are printed from left to right.

SET-B



Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [20 Points]

Write a Java program that takes an integer **N** as input, where **N contains at least 6 digits**.

Determine the **first (leftmost) digit** of the number. If the first digit is **even**, print **all the odd digits** of the number from **left to right**. If the first digit is **odd**, print **all the even digits** of the number from **left to right**.

Print the selected digits separated by a single space.

Sample Input	Sample Output	Explanation
82461357	1 3 5 7	The first digit is 8 , which is even. Therefore, all the odd digits are printed from left to right.
73184295	8 4 2	The first digit is 7 , which is odd. Therefore, all the even digits are printed from left to right.

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [20 Points]

A secret network system logs packet identifiers as a string of **lowercase letters**. During stable transmission, packet IDs appear in **continuous alphabetical order**, where each next character is exactly **one greater than the previous one**. Your task is to: Find the **longest continuous alphabetical sequence** in the string. If the **length of that sequence is greater than 5**, convert the sequence to **UPPERCASE** and print **every alternate character**, starting from the **first character**. If the length is **5 or less**, print the sequence normally (no skipping, no case change)

- You may only use `length()`, `charAt()`, loops, and conditions

Sample Input	Sample Output
xyzmnopqrstuvwxyz	MOQS UW
awdxyz	xyz

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [20 Points]

A DNA analysis system stores scan data as a string of **uppercase letters**. When the scanner works correctly, a **valid DNA signal** appears as a sequence of characters that follow **continuous alphabetical order**, meaning each next character is exactly one letter after the previous one. Find the **longest continuous alphabetical sequence** in the given string. If the **length of the sequence is smaller than 6**, convert the sequence to **lowercase** and print **every alternate character**, starting from the **first character** of the sequence. If the length is **6 or more**, print the sequence normally. (no skipping, no case change)

- You may only use `length()`, `charAt()`, loops, and conditions

Sample Input	Sample Output
XXMNOPQRSTUVWXYZ	MNOPQRSTUVWXYZ
ABXYZ	xz

Solution & Rubric

```
import java.util.Scanner;
class setA{
    public static void main(String[] args) {
        Scanner input=new Scanner(System.in);
        String st=input.nextLine();
        int currentStartIndex=0;
        int currentLength=1;
        int maxLength=0;
        int maxStartIndex=0;

        //identify the longest continuous alphabetical sequence
        for(int i =0; i<st.length()-1;i++){
            if(st.charAt(i)==(char) (st.charAt(i+1)-1)){
                currentLength++;
            }
            else {
                currentLength =1;
                currentStartIndex = i+1;
            }
            if(currentLength>maxLength ){
                maxLength=currentLength ;
                maxStartIndex=currentStartIndex ;
            }
        }

        //Correct conditional logic for output formatting (case change and skipping)
        int step=2;
        int caseShift=32;
        if(maxLength<=5){
            step=1; // no skipping
            caseShift=0; // no case change
        }
        for (int i = maxStartIndex ;i <maxStartIndex+maxLength; i=i+step) {
            char ch=st.charAt(i);
            ch=(char)(ch-caseShift);
            System.out.print(ch);

        }
    }
}
```

SET - [B]**SOLUTION**

```
import java.util.Scanner;

class setB {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        String st = input.nextLine();

        int currentStartIndex = 0;
        int currentLength = 1;
        int maxLength = 0;
        int maxStartIndex = 0;

        //identify the longest continuous alphabetical sequence
        for (int i = 0; i < st.length() - 1; i++) {
            if (st.charAt(i) == st.charAt(i + 1) - 1) {
                currentLength++;
            } else {
                currentLength = 1;
                currentStartIndex = i + 1;
            }

            if (currentLength > maxLength) {
                maxLength = currentLength;
                maxStartIndex = currentStartIndex;
            }
        }

        //Correct conditional logic for output formatting (case change and skipping)
        int step;
        int caseShift;
        if (maxLength < 6) {
            step = 2;
            caseShift = 32;
        } else {
            step = 1;    // no skipping
            caseShift = 0; // no case change
        }

        for (int i = maxStartIndex; i < maxStartIndex + maxLength; i += step) {
            char ch = st.charAt(i);
            ch = (char)(ch + caseShift);
            System.out.print(ch);
        }
    }
}
```



SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [14 Points]

In an online gaming lobby, player scores arrive live, one after another. The lobby can have a **maximum of 11 players**. Each time a new score is entered, the system must immediately display the **current Top-3 leaderboard**, showing the three highest scores entered so far. If **fewer than 3 scores** have been entered, the system should display the message **“Minimum 3 scores needed to see the leaderboard”** instead of showing any scores.

Score input continues until the user enters **-1** or until scores for **11 players** have been recorded, whichever occurs first. Once input ends, the system must display the **final Top-3 highest scores**.

Sample Input	Sample Output
40	Minimum 3 scores needed to see the leaderboard
55	Minimum 3 scores needed to see the leaderboard
20	Current Top 3: 55 40 20
70	Current Top 3: 70 55 40
65	Current Top 3: 70 65 55
-1	Final Top 3: 70 65 55

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [14 Points]

In a running race, athletes finish at different times, and each runner's **completion time (in seconds)** is recorded one by one. The race can have a **maximum of 11 runners**. After each new time is entered, the system must immediately display the **current 3 fastest completion times**, where a **lower time means a faster runner**.

If **fewer than 3 completion times** have been entered, the system should display the message **"Minimum 3 finish times required to show rankings"**. Time entry continues until the user enters **-1** or until **11 finish times** have been recorded, whichever occurs first. When input ends, the system must display the **final 3 fastest completion times**.

Sample Input	Sample Output
320	Minimum 3 finish times required to show rankings
305	Minimum 3 finish times required to show rankings
340	Current Fastest 3: 305 320 340
298	Current Fastest 3: 298 305 320
310	Current Fastest 3: 298 305 310
-1	Final Fastest 3: 298 305 310

Solution & Rubric

SOLUTION

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int[] scores = new int[11];
        int count = 0;

        while (count < 11) {

            int input = sc.nextInt();

            if (input == -1) {
                break;
            }

            scores[count] = input;
            count++;

            //Sorting
            for (int i = 0; i < count - 1; i++) {
                for (int j = 0; j < count - i - 1; j++) {
                    if (scores[j] > scores[j + 1]) {
                        int temp = scores[j];
                        scores[j] = scores[j + 1];
                        scores[j + 1] = temp;
                    }
                }
            }

            if (count < 3) {
                System.out.println("Minimum 3 scores needed to see the leaderboard");
            } else {
                System.out.println("Current Top 3: "
                    + scores[0] + " " + scores[1] + " " + scores[2]);
            }
        }

        if (count >= 3) {
            System.out.println("Final Top 3: "
                + scores[0] + " " + scores[1] + " " + scores[2]);
        }
    }
}
```

SET - [B]

SOLUTION

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int[] times = new int[11];
        int count = 0;

        while (count < 11) {

            int input = sc.nextInt();

            if (input == -1) {
                break;
            }

            times[count] = input;
            count++;

            // Sorting (Ascending)
            for (int i = 0; i < count - 1; i++) {
                for (int j = 0; j < count - i - 1; j++) {
                    if (times[j] > times[j + 1]) {
                        int temp = times[j];
                        times[j] = times[j + 1];
                        times[j + 1] = temp;
                    }
                }
            }

            if (count < 3) {
                System.out.println("Minimum 3 finish times required to show rankings");
            } else {
                System.out.println("Current Fastest 3: "
                    + times[0] + " " + times[1] + " " + times[2]);
            }
        }

        if (count >= 3) {
            System.out.println("Final Fastest 3: "
                + times[0] + " " + times[1] + " " + times[2]);
        }
    }
}
```

{ }